Интерфейс и кадр

Состояние, отрисовка и хранение данных



Frontend-разработчик

Что будем уметь

- Отличать императивное описание интерфейса от декларативного
- Выделять состояние приложения и единый источник правды
- Сравнивать стратегии обновления DOM, которые используют фреймворки
- Объяснять, как браузер рисует кадр и почему интерфейс подтормаживает
- Делать анимацию, не зависящую от частоты экрана
- Выбирать хранилище на клиенте и безопасно сохранять данные

Демонстрация после лекции: RPG «Жёлтый маг»

Где мы сейчас

Тема	Что уже знаем	Что добавим
03 DOM	менять элементы, события	как организовать много изменений
04 JavaScript	event loop, tasks, microtasks	где в цикле отрисовка кадра
04 JavaScript	объекты и ссылки	состояние и неизменяемость

Сегодня — от отдельных DOM-операций к устройству интерфейса

Императивный интерфейс

```
// Говорим, ЧТО СДЕЛАТЬ
const item = document.createElement("li");
item.textContent = "Искра";
list.append(item);

counter.textContent = Number(counter.textContent) + 1;
emptyHint.hidden = true;
```

Каждое изменение — отдельная команда DOM.

Порядок и полноту команд контролируем сами.

Итоговый экран зависит от всей истории команд

Декларативный интерфейс

```
// Описываем, КАК ДОЛЖНО БЫТЬ
function view(spells) {
  return `
    <p>Заклинаний: ${spells.length}</p>
    ${spells.length === 0 ? "<p>Книга пуста</p>" : ""}
    <ul>
      ${spells.map((s) => `<li>${s}</li>`).join("")}
    </ul>
  `;
}
```

Описываем экран для любых данных.

Как прийти к нему от текущего DOM — отдельная задача.

UI = f(state): экран — функция от данных

Императивно и декларативно

	Императивно	Декларативно
Что пишем	команды изменений	описание результата
Экран зависит от	истории действий	только текущих данных
Типичная ошибка	забыли обновить одно место	лишняя работа при обновлении
Где встречается	DOM API, jQuery	React, Vue, Svelte, Jetpack Compose

Декларативный подход переносит сложность из нашего кода в библиотеку

Состояние

```
const state = {
  mage: { hp: 30, maxHp: 30, mana: 20, maxMana: 20 },
  spells: [
    { id: "spark", name: "Искра", cost: 5 },
    { id: "shield", name: "Щит света", cost: 8 },
  ],
  isSpellbookOpen: false,
};
```

Данные, которые меняются со временем и определяют экран.

В состоянии нет элементов DOM.

Тест: по состоянию можно заново построить экран

Единый источник правды

```
const state = {
  spells: [
    { id: 1, name: "Искра" },
    { id: 2, name: "Щит" },
  ],
  selected: { id: 1, name: "Искра" },
};

state.spells[0].name = "Искра II";
console.log(state.selected.name);
```

Вывод:
Искра

selected — копия. Одно и то же заклинание хранится дважды.

Храним идентификатор, а не копию: selectedId: 1

Минимальное состояние

```
// Избыточно: три значения про одно и то же
const state = { spells, count: 2, hasSpells: true };

// Минимально
const state = { spells };
const count = state.spells.length;
const hasSpells = count > 0;
```

Что можно вычислить из состояния — не храним.

Вычисляем при отрисовке.

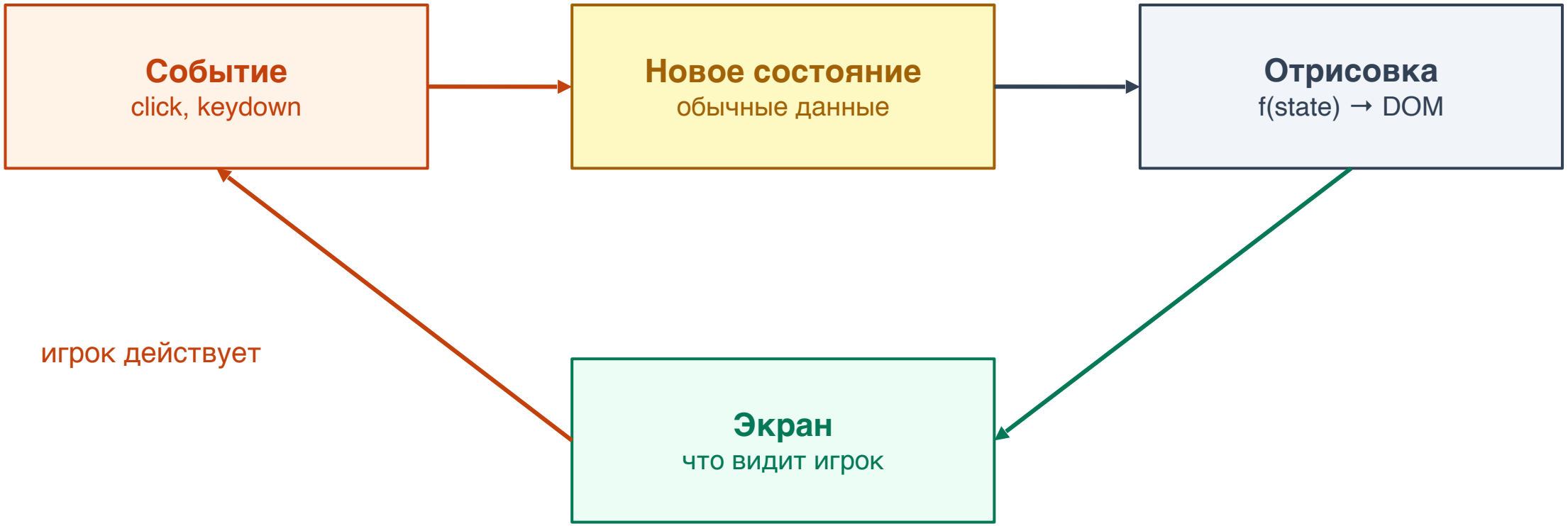
Производные данные вычисляем, иначе они разойдутся

Виды состояния

Вид	Пример	Кто хозяин
Данные приложения	HP, заклинания, уровень	наш state
Состояние интерфейса	открыта ли книга, вкладка	наш state
Состояние DOM	фокус, курсор в поле, прокрутка	браузер
Серверные данные	таблица рекордов	сервер, у нас копия
URL	?tab=spells	адресная строка

У каждого значения один хозяин

Однонаправленный поток данных



Данные текут в одну сторону: из состояния в DOM

Изменения как действия

```
function reduce(state, act) {
  switch (act.type) {
    case "cast":
      return { ...state, mana: state.mana - act.cost };
    case "rest":
      return { ...state, mana: state.maxMana };
    default:
      return state;
  }
}
let s = { mana: 20, maxMana: 20 };
s = reduce(s, { type: "cast", cost: 5 });
s = reduce(s, { type: "cast", cost: 5 });
console.log(s.mana);
```

Вывод:
10

Действие описывает, что произошло.

Все правила изменения — в одной функции.

Reducer: (состояние, действие) → новое состояние

Почему новый объект, а не изменение

```
const a = { mana: 20 };
const b = a;
b.mana = 15;
console.log(a === b, a.mana);

const prev = { mana: 20 };
const next = { ...prev, mana: 15 };
console.log(prev === next, prev.mana);
```

Вывод:
true 15
false 20

По ссылке не видно, что a изменился.

Новый объект — дешёвая проверка «изменилось ли» через ===

Состояние изменилось. Как обновить DOM?

Стратегия	Идея	Где встречается
Полная перерисовка	выбросить всё и построить заново	шаблоны, canvas
Точечные обновления	вручную менять изменившееся	ванильный JS, jQuery
Сравнение описаний	сравнить старое и новое описание, применить разницу	React, Preact
Реактивность	знать, кто от чего зависит, обновлять только их	Vue, Svelte, Solid

Все фреймворки решают одну задачу разными способами

Полная перерисовка

```
function render(state) {
  app.replaceChildren(
    SearchBox(state.query),
    SpellList(state.spells),
  );
}

app.addEventListener("input", (e) => {
  state.query = e.target.value;
  render(state);
});
```

Просто, рассинхронизация невозможна.

Но после каждой буквы поле создаётся заново: фокус пропадает.

Новые элементы теряют состояние DOM: фокус, ввод, прокрутку

Полная перерисовка и клики

```
// DOM перерисовывается каждый кадр (60 раз в секунду)

// mousedown — на старой кнопке
// через 16 мс кнопку заменили новой
// mouseup — уже на новой кнопке

// click получает их общий предок,
// а не кнопка
```

Нажали кнопку — ничего не произошло.

:hover и :active тоже мигают.

click срабатывает на общем предке элементов mousedown и mouseup

Точечные обновления

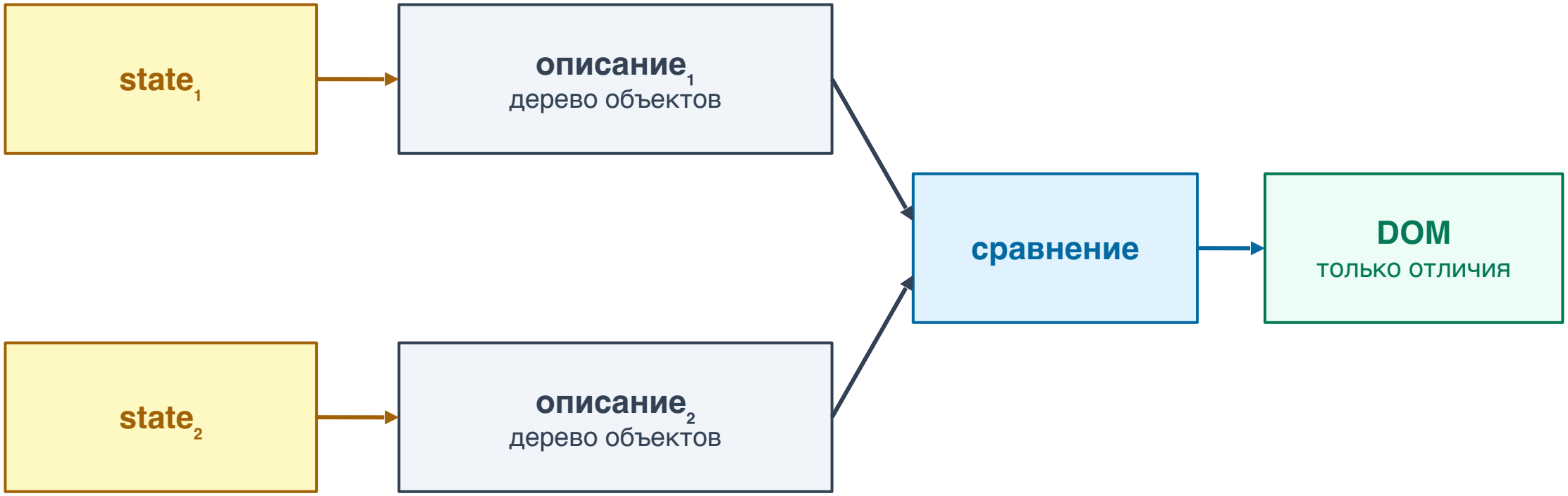
```
function update(prev, next) {
  if (prev.hp !== next.hp) updateHpBar(next);
  if (prev.mana !== next.mana) updateManaBar(next);
  if (prev.level !== next.level) {
    updateLevel(next);
    showLevelUp();
  }
  // + список, лог, щит, эффекты…
}
```

Быстро, состояние DOM сохраняется.

Но каждое поле сравниваем вручную.

Точно и быстро, но код растёт с каждым полем

Сравнение описаний



Пишем как полную перерисовку, DOM меняется точечно

Описание — обычный объект

```
// описание кнопки, а не сам DOM-элемент
const vnode = {
  type: "button",
  props: { disabled: false },
  children: ["Искра (5)"],
};

// JSX в React превращается в похожие объекты
// <button disabled={false}>Искра (5)</button>
```

Создать объект дёшево.

Сравнить два дерева объектов дешевле, чем трогать DOM.

Virtual DOM — дерево обычных JavaScript-объектов

Списки и ключи

Было: [Искра, Щит, Луч]
Стало: [Щит, Луч]

По позиции:
  0: «Искра» → «Щит»
  1: «Щит» → «Луч»
  2: удалить

По ключу:
  удалить элемент с key="spark"

Без ключа сравнение идёт по позиции: три изменения вместо одного.

Состояние элементов «переезжает» к чужим данным.

Ключ — устойчивый id элемента списка, а не его индекс

Реактивность: сигналы

```
function signal(value) {
  const subs = new Set();
  return {
    get: () => value,
    set(next) {
      value = next;
      subs.forEach((fn) => fn(value));
    },
    subscribe: (fn) => subs.add(fn),
  };
}
const mana = signal(20);
mana.subscribe((v) => (manaText.textContent = v));
mana.set(15); // обновится только manaText
```

Значение знает своих подписчиков.

Деревья не сравниваем: обновляется только то, что зависит от изменения.

Сигналы: зависимости известны заранее

Сравнение стратегий

	Полная	Точечная	Сравнение	Реактивность
Наш код	простой	сложный	простой	простой
Состояние DOM	теряется	сохраняется	сохраняется	сохраняется
Лишняя работа	вся страница	нет	сравнение деревьев	почти нет
Рассинхронизация	нет	частая	нет	нет

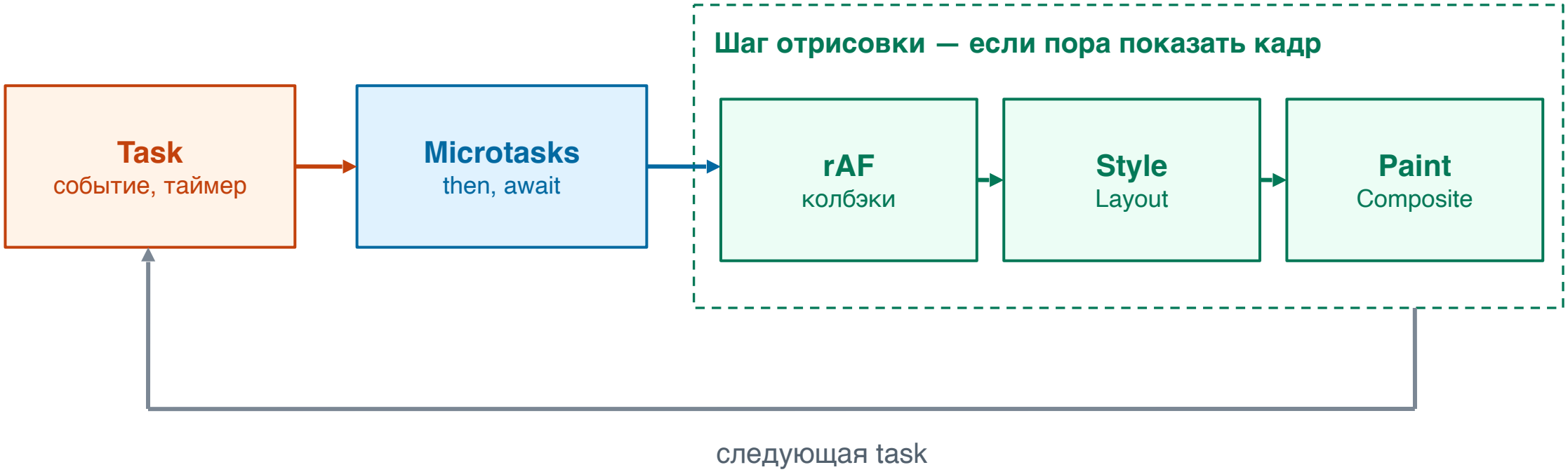
Простоту полной перерисовки и точность ручной даёт библиотека

Частота экрана и бюджет кадра

Экран	Кадров в секунду	Бюджет на кадр
60 Гц	60	16,7 мс
90 Гц	90	11,1 мс
120 Гц	120	8,3 мс
144 Гц	144	6,9 мс

JavaScript, стили, layout и paint должны уложиться в бюджет

Где в event loop отрисовка



Браузер рисует между задачами, никогда посреди JavaScript

Пока работает JavaScript, кадра нет

```
button.addEventListener("click", () => {
  status.textContent = "Колдую…";

  const end = Date.now() + 2000;
  while (Date.now() < end) {} // 2 секунды занятости

  status.textContent = "Готово";
});
```

«Колдую…» на экране не появится.

Кадр нарисуется после обработчика — уже с «Готово».

Изменения DOM видны после завершения текущей задачи

Долгие задачи

```
new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    const ms = Math.round(entry.duration);
    console.log("Долгая задача:", ms, "мс");
  }
}).observe({ type: "longtask", buffered: true });
```

Задача дольше 50 мс — long task.

Пока она идёт, страница не отвечает и не рисует.

Долгую работу дробим или выносим в Web Worker

Таймер или requestAnimationFrame

	setTimeout / setInterval	requestAnimationFrame
Когда вызывается	после задержки, когда поток свободен	перед отрисовкой кадра
Связь с экраном	нет	один вызов на кадр
Фоновая вкладка	замедляется	обычно не вызывается
Аргумент	нет	время кадра

Для анимации — requestAnimationFrame

requestAnimationFrame

```
let x = 0;

function frame() {
  x += 5;
  spark.style.transform = `translateX(${x}px)`;
  if (x < 600) requestAnimationFrame(frame);
}

requestAnimationFrame(frame);
```

Колбэк выполнится перед следующей отрисовкой.

Чтобы продолжить, снова запрашиваем кадр.

rAF: «вызови меня перед следующим кадром»

Скорость зависит от экрана

Экран	Кадров в секунду	Путь за секунду при x += 5
60 Гц	60	300 px
120 Гц	120	600 px
144 Гц	144	720 px

Шаг «за кадр» делает скорость зависимой от железа

Delta time

```
const SPEED = 300; // пикселей в секунду
let x = 0;
let last = performance.now();

function frame(now) {
  const dt = (now - last) / 1000; // секунды
  last = now;
  x += SPEED * dt;
  spark.style.transform = `translateX(${x}px)`;
  requestAnimationFrame(frame);
}

requestAnimationFrame(frame);
```

rAF передаёт время кадра.

Смещение = скорость × прошедшее время.

Скорость задаём в единицах в секунду, а не за кадр

Фиксированный шаг

```
const STEP = 1 / 60; // секунды
let acc = 0;

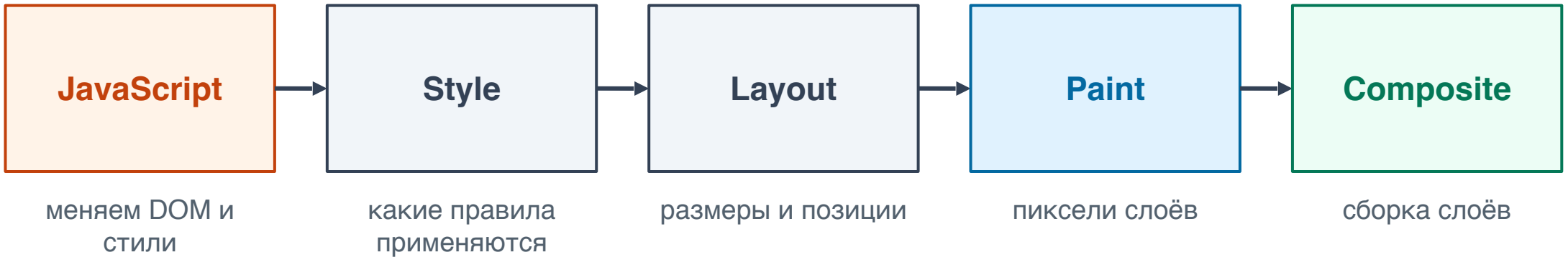
function frame(now) {
  acc += Math.min((now - last) / 1000, 0.25);
  last = now;
  while (acc >= STEP) {
    update(state, STEP); // всегда один и тот же шаг
    acc -= STEP;
  }
  render(state);
  requestAnimationFrame(frame);
}
```

Симуляция идёт одинаковыми шагами при любой частоте кадров.

Результат воспроизводим.

Переменный шаг — проще, фиксированный — стабильнее физика

Как браузер рисует кадр



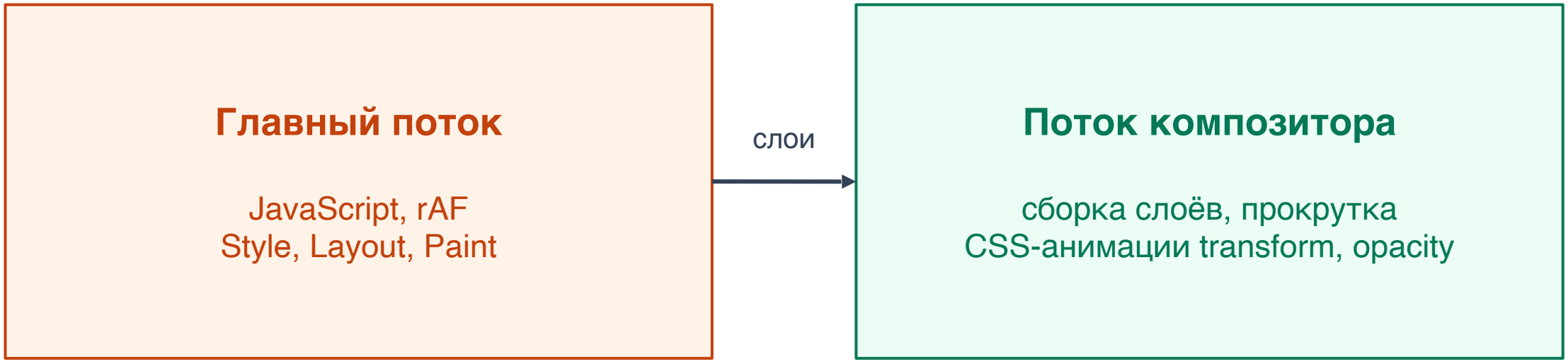
Чем раньше в цепочке изменение, тем больше работы после него

Что запускает каждое изменение

Меняем	Что браузер делает заново
width, height, top, left, margin	Layout → Paint → Composite
color, background, box-shadow	Paint → Composite
transform, opacity	обычно только Composite

Двигаем и масштабируем через transform

Главный поток и поток композитора



Главный поток занят долгой задачей → JS-анимации и клики стоят, CSS-анимация transform продолжается

Декоративную анимацию отдаём CSS, игровую логику — rAF

Анимация на CSS

```
@keyframes float {
  from { transform: translateY(0); }
  to   { transform: translateY(-12px); }
}

.mage-portrait {
  animation: float 1s ease-in-out infinite alternate;
}
```

Ни строчки JavaScript.

Браузер сам решает, где выполнять анимацию.

Если анимация не зависит от логики — CSS

Принудительный layout

```
// Плохо: чтение и запись вперемешку
for (const bar of bars) {
  const width = bar.parentElement.offsetWidth; // layout!
  bar.style.width = `${width / 2}px`;
}

// Лучше: сначала все чтения, потом все записи
const widths = bars.map((b) => b.parentElement.offsetWidth);
bars.forEach((bar, i) => {
  bar.style.width = `${widths[i] / 2}px`;
});
```

Чтение размера после записи — немедленный пересчёт layout. Группируем чтения и записи

DOM или Canvas

	DOM	Canvas 2D
Модель	браузер хранит дерево	только пиксели
Текст, доступность	из коробки	вручную
Клик по объекту	событие элемента	считаем попадание сами
Сотни движущихся объектов	тяжело	подходит
Подходит для	интерфейса, клеточных игр	арен, частиц

Можно сочетать: арена на canvas, интерфейс в DOM

Где хранить данные в браузере

	Объём	API	Особенности
Cookies	~4 КБ	строка	уходят на сервер с каждым запросом
localStorage	~5 МБ	синхронный, строки	хранится, пока не очистят
sessionStorage	~5 МБ	синхронный, строки	до закрытия вкладки
IndexedDB	сотни МБ и больше	асинхронный, объекты	большие данные, офлайн

Сохранение игры — localStorage, большие данные — IndexedDB

Хранилище принадлежит origin

origin = схема + хост + порт

```
https://mage.example         ← одно хранилище
https://mage.example/battle  ← то же самое
http://mage.example          ← другое: схема
http://localhost:5173        ← другое
http://localhost:4173        ← и это другое
```

Сайты не видят данные друг друга.

Даже разные порты на localhost — разные хранилища.

Хранилище привязано к origin, а не к сайту в целом

localStorage

```
const SAVE_KEY = "yellow-mage:save";

function save(mage) {
  const json = JSON.stringify({ level: mage.level });
  localStorage.setItem(SAVE_KEY, json);
}

function load() {
  const raw = localStorage.getItem(SAVE_KEY);
  return raw === null ? null : JSON.parse(raw);
}
```

Хранит только строки.

Объект превращаем в JSON и обратно.

Вызов синхронный.

Сохраняем данные, а не элементы DOM

JSON и structuredClone

```
const save = {
  savedAt: new Date(2026, 9, 20),
  spells: new Set(["Искра"]),
};

const viaJson = JSON.parse(JSON.stringify(save));
const viaClone = structuredClone(save);

console.log(typeof viaJson.savedAt, viaJson.spells);
console.log(viaClone.savedAt instanceof Date,
  viaClone.spells.has("Искра"));
```

Вывод:
string {}
true true

JSON теряет Date и Set.

structuredClone их сохраняет.

localStorage — через JSON, IndexedDB — через structured clone

Загрузке не доверяем

```
function loadSave() {
  try {
    const raw = localStorage.getItem(SAVE_KEY);
    const data = JSON.parse(raw);
    if (typeof data?.level !== "number") return null;
    return data;
  } catch {
    return null; // повреждённая строка
  }
}
```

Сохранение могло остаться от старой версии или быть изменено в DevTools.

Данные из хранилища проверяем как пользовательский ввод

Версия формата сохранения

```
const SAVE_VERSION = 2;

function migrate(data) {
  if (data.version === 1) {
    // в версии 1 мана не сохранялась
    data = { ...data, mana: 20, version: 2 };
  }
  return data.version === SAVE_VERSION ? data : null;
}

console.log(migrate({ version: 1, level: 3 }));
console.log(migrate({ version: 7, level: 3 }));
```

Вывод:
{ version: 2, level: 3, mana: 20 }
null

Игра обновится, а старые сохранения останутся.

Сохранение с номером версии можно перенести в новый формат

Когда сохранять

```
function onVictory(state) {
  state.mage.xp += 15;
  save(state.mage);
}

document.addEventListener("visibilitychange", () => {
  if (document.visibilityState === "hidden") {
    save(state.mage);
  }
});
```

Не каждый кадр: setItem синхронный.

Сохраняем на важных событиях и при уходе со страницы.

Сохранение — по событию, а не по кадру

Мини-задача: что покажет label

```
const state = { mana: 10 };

button.addEventListener("click", () => {
  state.mana -= 5;

  requestAnimationFrame(() => {
    label.textContent = state.mana;
  });

  state.mana -= 5;
});
```

Кнопку нажали один раз.

Что будет в label после следующего кадра?

Подумать 30 секунд, ответ на следующем слайде

Разбор: 0

```
// 1. click: mana = 5
// 2. rAF регистрирует колбэк на следующий кадр
// 3. click продолжается: mana = 0
// 4. обработчик завершён, microtasks пусты
// 5. шаг отрисовки: колбэк читает state.mana → 0
```

Колбэк хранит доступ к state, а не снимок значения.

Колбэк читает состояние в момент вызова, а не регистрации

Мини-задача: сколько изменений DOM

Список заклинаний, у элементов нет ключей.

Было: [Искра, Щит, Луч, Метеор]
Стало: [Щит, Луч, Метеор]

Сколько элементов изменит сравнение по позиции?
А сколько — сравнение по ключу?

Подсчитать изменения текста и удаления элементов.

Подумать 30 секунд

Разбор: 4 против 1

По позиции:
0: «Искра» → «Щит»
1: «Щит» → «Луч»
2: «Луч» → «Метеор»
3: удалить

По ключу:
удалить элемент с key="spark"

Удаление первого элемента сдвигает все позиции.

Ключ позволяет узнать элементы на новых местах.

По позиции: 3 изменения + 1 удаление. По ключу: 1 удаление

Демонстрация после лекции

- Жёлтый маг: состояние, отрисовка из компонентов-функций
- Поле поиска: потеря фокуса при полной перерисовке
- Клик, пропадающий при перерисовке DOM каждый кадр
- Арена на canvas: игровой цикл на rAF без dt и с dt
- Сохранение и загрузка прогресса из localStorage

Проект yellow-mage: npm install, затем npm run dev

Что важно унести с собой

- Экран — функция от состояния, данные текут в одну сторону
- У каждого факта одно место хранения, производное вычисляем
- Фреймворки по-разному решают одну задачу: обновить DOM точечно
- Кадр рисуется между задачами, бюджет — 16,7 мс и меньше
- Анимация: rAF и скорость в единицах в секунду; декоративная — CSS
- localStorage — JSON с версией, при загрузке проверяем

Следующая лекция: модули, пакеты и сборка

Источники и дальнейшее чтение

- react.dev: Thinking in React, Choosing the State Structure, Render and Commit
- Redux: State, Actions and Reducers; документация Solid и Vue по реактивности
- WHATWG HTML: update the rendering; MDN: requestAnimationFrame
- web.dev: Rendering performance, RAIL, Optimize long tasks
- Glenn Fiedler: Fix Your Timestep!
- MDN: Web Storage, Storage quotas, structuredClone

Ссылки на конкретные разделы находятся в заметках слайдов